

סיכום למבחן --- דאטה סיינס וסייבר

ד"ר אורי איתי

Analysis Data Exploratory --- EDA

EDA הוא השלב הראשון בכל פרויקט דאטה סיינס.
מטרותיו:

- הבנת הנתונים.
- זיהוי ערכים חסרים.
- זיהוי חריגים.
- הבנת התפלגויות.
- איתור קשרים בין משתנים.
- בדיקת איכות הנתונים.

חשוב לזכור:

- **מודל טוב אינו יכול לפצות על נתונים גרועים.** איכות הנתונים, תהליך האיסוף והבנת הדאטה חשובים לעיתים יותר מבחירת האלגוריתם עצמו.
- **אולי אינך מתעניין בסייבר, אך הסייבר בהחלט מתעניין בך.** כל מערכת האוספת, מעבדת או שומרת מידע עלולה להפוך למטרה, גם אם אבטחת מידע אינה המטרה המרכזית של הפרויקט.
- **מודל שאינו מסייע בהגנה על הארגון הוא מודל לא טוב.** במערכות סייבר הערך האמיתי של המודל נמדד ביכולת שלו לסייע בקבלת החלטות, לזהות איומים ולהקטין סיכון.
- **בסופו של דבר, המטרה היא לתפוס את ההתקפה.** אם פתרון פשוט מצליח לזהות את האיום בצורה אמينة, הוא עשוי להיות עדיף על מודל מורכב שקשה להסביר, לתחזק או להפעיל.
- **הבנת הבעיה חשובה לא פחות מהבנת האלגוריתם.** לפני שבחרים מודל, יש להבין מה מנסים לזהות, מהו הנזק האפשרי ומהם האילוצים העסקיים והמבצעיים.
- **לא כל חריגה היא התקפה, ולא כל התקפה נראית חריגה.** אחד האתגרים המרכזיים בסייבר הוא להבחין בין התנהגות חריגה לגיטימית לבין פעילות זדונית אמיתית.
- **Positive False מעייף את הארגון, Negative False מסכן אותו.** לכן יש לבחור את מדדי ההערכה בהתאם למשימה ולא להסתפק ב-Accuracy בלבד.
- **EDA טוב חוסך שעות רבות של אימון מודלים.** לעיתים תובנה אחת המתקבלת מחקירת הנתונים שווה יותר מעשרות ניסויים באלגוריתמים שונים.
- **המודל הטוב ביותר אינו תמיד המודל המדויק ביותר.** לעיתים מודל מעט פחות מדויק אך פשוט, יציב, בטוח, ומוסבר היטב יהיה הבחירה הנכונה.
- **בסייבר, זמן תגובה הוא חלק מהפתרון.** מודל מושלם שמחזיר תשובה לאחר שעה עשוי להיות פחות שימושי ממודל פשוט שמתריע תוך שניות.

□ **ההאקר צריך להצליח פעם אחת. המגן צריך להצליח בכל פעם.** לכן ניהול סיכונים, ניטור מתמשך ושכבות הגנה חשובים לא פחות ממודל החיזוי עצמו.

□ **כל מודל שגוי. חלק מהמודלים שימושיים.** (All models are wrong, but some are useful) --- George Box) המטרה אינה לבנות מודל מושלם, אלא מודל המסייע להבין את המציאות ולקבל החלטות טובות יותר.

קורלציות

Correlation Pearson

מודד קשר ליניארי בין משתנים.

$$-1 \leq \rho \leq 1$$

מתאים כאשר:

□ המשתנים רציפים.

□ הקשר ליניארי.

רגיש לערכים חריגים.

Correlation Spearman

מבוסס על דירוגים.

מודד קשר מונוטוני.

יתרונות:

□ פחות רגיש לחריגים.

□ מתאים לקשרים שאינם ליניאריים.

Correlation Kendall

מודד התאמה בין סדרי דירוג.

מתאים:

□ למדגמים קטנים.

□ כאשר קיימים ערכים זהים רבים.

בחירת שיטת קורלציה

שיטה	מצב
Pearson	קשר ליניארי
Spearman	קשר מונוטוני
Kendall	מדגם קטן

CrossTab

CrossTab היא טבלת שכיחויות דו-ממדית.
דוגמה:

<i>Country</i>	<i>Admin</i>	<i>User</i>
<i>Israel</i>	100	900
<i>USA</i>	80	700

שימושים:

□ זיהוי קשרים בין משתנים.

□ זיהוי היררכיות.

□ זיהוי. Aliasing.

□ זיהוי צירופים חריגים.

Detection Alias

לעיתים אותו אדם מופיע תחת מספר שמות משתמש.

<i>Username</i>	<i>Email</i>
<i>john1</i>	<i>a@x.com</i>
<i>john2</i>	<i>a@x.com</i>

CrossTab יכולה לחשוף מקרים כאלה.

היררכיות

דוגמאות:

□ מדינה → עיר.

□ פקולטה → מחלקה.

□ מנהל → עובד.

בדרך כלל כל ילד משויך להורה יחיד.

אנומליות

אנומליה חד-ממדית

ערך חריג במשתנה יחיד.
לדוגמה:

קובץ בגודל 100GB כאשר רוב הקבצים בגודל 1MB.

אנומליה רב־ממדית

כל משתנה תקין בנפרד אך השילוב חריג.
לדוגמה:

□ התחברות מחו"ל.

□ בשעה 03:00.

□ הורדת אלפי קבצים.

שיטות לזיהוי אנומליות

Z-Score

$$Z = \frac{x - \mu}{\sigma}$$

מתאים בעיקר לדאטה נורמלי.

IQR

$$IQR = Q_3 - Q_1$$

פחות רגיש לחריגים.

Deviation Absolute Median --- MAD

MAD (Median Absolute Deviation) הוא מדד לפיזור הנתונים המבוסס על החציון במקום על הממוצע.
הוא פותח כחלופה עמידה (Robust) לסטיית התקן.

הגדרה

נתונה סדרת נתונים:

$$x_1, x_2, \dots, x_n$$

ראשית מחשבים את החציון:

$$M = \text{Median}(X)$$

לאחר מכן מחשבים את המרחק המוחלט של כל ערך מהחציון:

$$|x_i - M|$$

לבסוף מחשבים את החציון של המרחקים:

$$MAD = \text{Median}(|x_i - M|)$$

דוגמה

נתונים:

1, 2, 3, 4, 5

החציון הוא:

$$M = 3$$

המרחקים המוחלטים:

2, 1, 0, 1, 2

החציון שלהם:

$$MAD = 1$$

דוגמה עם חריג

נתונים:

1, 2, 3, 4, 1000

החציון עדיין:

$$M = 3$$

המרחקים:

2, 1, 0, 1, 997

ולכן:

$$MAD = 1$$

שימו לב:

הערך החריג כמעט ואינו משפיע על MAD.

השוואה לסטיית תקן

סטיית התקן:

$$\sigma = \sqrt{\frac{1}{n} \sum (x_i - \mu)^2}$$

מבוססת על:

□ ממוצע.

□ ריבועי מרחקים.

לכן היא רגישה מאוד לחריגים.

לעומת זאת:

MAD מבוסס על:

□ חציון.

□ מרחקים מוחלטים.

ולכן הוא עמיד הרבה יותר.

MAD לזיהוי אנומליות

ניתן להגדיר:

$$\text{Modified } Z = 0.6745 \frac{x_i - M}{MAD}$$

באופן דומה ל-Z-Score.

כלל נפוץ:

אם:

$$|\text{Modified } Z| > 3.5$$

הנקודה נחשבת חריגה.

מדוע MAD חשוב בסייבר?

נתוני סייבר רבים מאופיינים ב:

□ זנבות כבדים.

□ התפלגויות מוטות.

□ ערכים קיצוניים.

□ מתקפות נדירות.

לדוגמה:

□ מספר הורדות קבצים.

□ מספר חיבורים.

□ נפח תעבורה.

□ מספר ניסיונות התחברות.

במקרים כאלה:

□ הממוצע עלול להיות מטעה.

□ סטיית התקן עלולה להיות מנופחת.

□ MAD מספק אומדן יציב יותר.

השוואה

מדד	מבוסס ממוצע	עמיד לחריגים
Deviation Standard	כן	לא
MAD	לא	כן

נקודה למבחן

MAD הוא המקביל ה"רובסטי" של סטיית התקן.
כאשר קיימים:

- חריגים רבים.
- התפלגות מוטה.
- זנבות כבדים.

לעיתים עדיף להשתמש ב-MAD במקום בסטיית התקן לצורך זיהוי אנומליות וניתוח הנתונים.

Forest Isolation

הרעיון:
אנומליות מבודדות במהירות בעצי החלטה.
יתרונות:

- מתאים לדאטה רב-ממדי.
- אינו מניח התפלגות מסוימת.

LOF

בודק צפיפות מקומית.
מתאים כאשר החריגות מקומיות.

Explainability

יכולת להסביר את החלטות המודל.
חשובה במיוחד:

- ברפואה.
- בפיננסים.
- בסייבר.

מדוע יתירות פוגעת בהסבריות?

כאשר מספר משתנים מכילים מידע דומה:

- החשיבות מתחלקת ביניהם.
- קשה להבין מי באמת משפיע.

Engineering Feature

יצירת משתנים חדשים או שינוי משתנים קיימים.
דוגמאות:

Hour. Per Logins Failed □

Size. Download Average □

User. Per Countries Of Number □

לעיתים שלב זה חשוב יותר מבחירת המודל.

טרנספורמציות

Transform Log

$$x' = \log(x)$$

שימושי כאשר קיימת הטיה חזקה.

Box-Cox

משפחה של טרנספורמציות שמטרתן:

Skewness. הקטנת □

□ ייצוב שונות.

סדרות זמן

דוגמאות:

□ ניסיונות התחברות.

□ IDS. התראות.

□ מתקפות ביום.

השכיח וכיסוי ההתפלגות

השכיח (Mode) הוא הערך המופיע בתדירות הגבוהה ביותר בנתונים.
נסמן:

n

כמספר הרשומות הכולל,
-1

$f(x)$

כתדירות של הערך x .
השכיח מוגדר:

$$Mode = \arg \max_x f(x)$$

אחוז הכיסוי של השכיח

לעיתים מעניין לא רק לדעת מהו השכיח, אלא גם כמה מהדאטה הוא מסביר.
אם השכיח מופיע:

$$f_{max}$$

פעמים,
אז אחוז הכיסוי שלו הוא:

$$Coverage_1 = \frac{f_{max}}{n}$$

לדוגמה:

שכיחות	דפדפן
700	Chrome
200	Firefox
80	Edge
20	Safari

סה"כ:

$$n = 1000$$

ולכן:

$$Coverage_1 = \frac{700}{1000} = 70\%$$

כלומר השכיח לבדו מסביר 70% מהנתונים.

K הערכים הנפוצים ביותר

לעיתים השכיח לבדו אינו מספיק.
נמדין את הערכים לפי שכיחות:

$$f_1 \geq f_2 \geq f_3 \geq \dots$$

כיסוי K הערכים הנפוצים ביותר מוגדר:

$$Coverage_K = \frac{\sum_{i=1}^K f_i}{n}$$

לדוגמה:

$$700, 200, 80, 20$$

עבור:

$$K = 2$$

נקבל:

$$Coverage_2 = \frac{700 + 200}{1000} = 90\%$$

כלומר שני הערכים הנפוצים ביותר מסבירים 90% מהדאטה.

מספר הערכים המינימלי המכסה $P\%$ מהדאטה

לעיתים רוצים לענות על השאלה:

כמה ערכים שונים דרושים כדי להסביר $P\%$ מהנתונים?

נסמן:

$$P$$

כאחוז הכיסוי הרצוי.

נחפש את:

$$K^* = \min \left\{ K : \frac{\sum_{i=1}^K f_i}{n} \geq P \right\}$$

כלומר:

המספר הקטן ביותר של ערכים המכסה לפחות $P\%$ מהדאטה.

דוגמה

נניח:

$$700, 200, 80, 20$$

ונרצה:

$$P = 95\%$$

נחשב:

$$Coverage_1 = 70\%$$

$$Coverage_2 = 90\%$$

$$Coverage_3 = 98\%$$

ולכן:

$$K^* = 3$$

כלומר שלושה ערכים בלבד מסבירים 98% מהדאטה.

מדוע זה חשוב?

ניתוח זה מאפשר להבין:

- עד כמה ההתפלגות מרוכזת.
- האם קיימים ערכים דומיננטיים.
- האם קיימת התפלגות מסוג Law. Power
- האם ניתן לבצע דחיסה של קטגוריות נדירות.
- האם יש מקום לבניית סגמנטים.

יישומים בסייבר

דוגמאות:

- מספר קטן של כתובות IP אחראי לרוב התעבורה.
 - מספר קטן של משתמשים אחראי לרוב הפעילות.
 - מספר קטן של פקודות אחראי לרוב הלוגים.
 - מספר קטן של דומיינים אחראי לרוב החיבורים.
- כאשר K^* קטן מאוד יחסית למספר הערכים האפשריים, הדבר עשוי להעיד על:
- ריכוזיות גבוהה.
 - התפלגות Heavy-Tailed.
 - פעילות חריגה.
 - קיומם של ים-Hub ברשת.

נקודה למבחן

השכיח מספק מידע על הערך הנפוץ ביותר, אך לעיתים חשוב יותר לבדוק:

- כמה מהדאטה הוא מכסה.
 - כמה ערכים נדרשים כדי להסביר את רוב הנתונים.
 - האם מספר קטן של ערכים שולט בהתפלגות.
- ניתוח זה מהווה כלי חשוב ב-EDA, בניתוח סגמנטים, ובזיהוי התפלגויות Law Power וחריגות בעולם הסייבר.

Series Time Slow

לדוגמה:

□ מתקפות כופרה.

□ פריצות חמורות.

כאשר יש מעט אירועים:

□ קשה ללמוד מחזוריות.

□ קשה לבצע חיזוי.

ARIMA

מודל קלאסי לחיזוי סדרות זמן.

LSTM

רשת נוירונים לסדרות זמן.
דורשת בדרך כלל כמות גדולה של נתונים.

פרטיות

מזהים ישירים

□ שם.

□ תעודת זהות.

□ מספר דרכון.

Identifiers Quasi

□ גיל.

□ מין.

□ מיקוד.

K-Anonymity

הרעיון:

כל רשומה אינה נבדלת מלפחות $K - 1$ רשומות נוספות.

מטרה:

הקטנת הסיכון לזיהוי אדם.

מגבלה

ייתכן שכל הרשומות בקבוצה חולקות אותו מידע רגיש.

Privacy Differential

הרעיון:
הוספת רעש אקראי לתוצאות.
המטרה:
הגנה על פרטיות המשתמשים.

NLP וסייבר

NLP מאפשר:

- זיהוי פשינג.
- ניתוח הודעות דואר.
- ניתוח לוגים.
- ניתוח Intelligence, Threat

Analytics Graph

גרף מורכב מ:

- קודקודים (Vertices).
- קשתות (Edges).

שימושים:

- זיהוי קהילות.
- זיהוי בוטנטים.
- זיהוי תוקפים.
- זיהוי נתיבי תקיפה.

Attacks Adversarial

הרעיון:
שינוי קטן בקלט גורם למודל לטעות.
הקלט נראה תקין לאדם אך מטעה את המודל.

מדדי הערכה

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

מודד כמה מההתראות היו נכונות.

Recall

$$Recall = \frac{TP}{TP + FN}$$

מודד כמה מהתקיפות זוהו.

Score F1

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

מדוע Accuracy עלול להטעות?

אם 99.9% מהמערכת תקינה, מודל שחוזר תמיד ``תקין'' יקבל Accuracy גבוה מאוד אך יהיה חסר ערך.

טיפ אחרון

כאשר נשאלת שאלה במבחן, נסו תמיד להסביר:

1. מהו הרעיון.
 2. מתי משתמשים בו.
 3. מהם היתרונות.
 4. מהם החסרונות.
 5. כיצד הוא קשור לעולם הסייבר.
- זו בדרך כלל הדרך לקבל את מלוא הניקוד.

סיסמאות והתקפות נפוצות

סיסמאות הן אחד ממנגנוני ההגנה הנפוצים ביותר במערכות מחשב. כתוצאה מכך, תוקפים רבים מנסים לתקוף אותן באופן ישיר או עקיף.

Attack Dictionary

בהתקפת מילון (Dictionary Attack) התוקף מנסה רשימת סיסמאות מוכנות מראש הכוללת:

- מילים נפוצות.
- שמות פרטיים.
- סיסמאות נפוצות.
- וריאציות פשוטות של מילים.

דוגמאות:

password □

123456 □

qwerty □

admin123 □

יתרון:

- מהיר יחסית.

חסרון:

- אינו מסוגל למצוא סיסמאות מורכבות שאינן במילון.

Attack Force Brute

בהתקפת כוח גס (Brute Force Attack) התוקף מנסה את כל הצירופים האפשריים.
לדוגמה:

aaa □

aab □

aac □

... □

יתרון:

- מובטח למצוא את הסיסמה בסופו של דבר.

חסרון:

- יקר מאוד מבחינת זמן חישוב.

Stuffing Credential

בהתקפה זו התוקף משתמש ברשימות של:

□ שמות משתמש.

□ כתובות דוא"ל.

□ סיסמאות שנגנבו בעבר.

ומנסה אותן במערכות אחרות.

הסיבה להצלחת ההתקפה היא שמשתמשים רבים עושים שימוש חוזר באותה סיסמה.

Tables Rainbow

טבלת קשת (Rainbow Table) היא טבלה המכילה:

□ סיסמאות אפשריות.

□ ערכי Hash מתאימים.

במקום לחשב את פונקציית הגיבוב בכל פעם, התוקף מבצע חיפוש בטבלה.
הרעיון:

$$Password \rightarrow Hash$$

לדוגמה:

$$password123 \rightarrow 482c811da5d5b4bc6d497ffa98491e38$$

אם ערך ה-Hash דלף ממסד הנתונים, ניתן לעיתים למצוא את הסיסמה המקורית באמצעות הטבלה.

Salt

כדי להתגונן מפני Tables Rainbow מוסיפים ערך אקראי הנקרא:

$$Salt$$

במקום לחשב:

$$Hash(password)$$

מחשבים:

$$Hash(password + salt)$$

כך שתי סיסמאות זהות יקבלו Hash שונה.

Functions Hash

פונקציות גיבוב נפוצות:

MD5 □

SHA1 □

SHA256 □

SHA512 □

כיום:

MD5 נחשב שבור. □

SHA1 נחשב חלש. □

SHA256 ו-SHA512 עדיין נפוצים. □

Storage Password

לא שומרים סיסמאות בצורה גלויה.
נהוג לשמור:

$$\text{Hash}(\text{password} + \text{salt})$$

לעיתים משתמשים גם בפונקציות ייעודיות:

bcrypt □

scrypt □

Argon2 □

המאטות את תהליך החישוב ומקשות על התוקף לבצע Force. Brute

מלכודות דבש --- Honeypots

מלכודת דבש (Honeypot) היא מערכת, שירות או משאב מחשוב המוקמים במכוון כדי למשוך תוקפים ולאפשר לארגון ללמוד את שיטות הפעולה שלהם. הרעיון המרכזי הוא שהמערכת אינה מיועדת לשימוש אמיתי. לכן, כל גישה אליה נחשבת חשודה מלכתחילה.

מטרות השימוש במלכודות דבש

□ זיהוי תוקפים וניסיונות חדירה.

□ איסוף מידע על טכניקות תקיפה.

□ זיהוי כלי תקיפה חדשים.

□ יצירת התראה מוקדמת על פעילות זדונית.

□ הסחת דעת של התוקף ממערכות אמיתיות.

סוגי מלכודות דבש

Honeypot Interaction Low

מדמה מספר שירותים בסיסיים בלבד.
יתרונות:

- קל להקמה.
- סיכון נמוך.
- צריכת משאבים נמוכה.

חסרונות:

- מספק מעט מידע על התוקף.
- תוקף מנוסה עשוי לזהות שמדובר במלכודת.

Honeypot Interaction High

מערכת מלאה המדמה סביבת עבודה אמיתית.
יתרונות:

- מספקת מידע רב על התוקף.
- מאפשרת לימוד מתקדם של טכניקות תקיפה.

חסרונות:

- מורכבת לניהול.
- עלולה להיות מנוצלת כנקודת זינוק לתקיפות אחרות אם אינה מבודדת היטב.

Honeytokens

לעיתים אין צורך במערכת שלמה.
ניתן ליצור "פיתיונות" דיגיטליים כגון:

- קובץ סיסמאות מזויף.
- מספר כרטיס אשראי פיקטיבי.
- משתמש דמה.
- מסמך מסווג לכאורה.

גישה אליהם מהווה אינדיקציה חזקה לפעילות זדונית.

דוגמה

נניח ששרת מכיל תיקייה בשם:

Executive_Passwords.xlsx

אף עובד לגיטימי אינו אמור לגשת לקובץ זה.

לכן כל ניסיון קריאה או העתקה של הקובץ עשוי להפעיל התראה מיידית.

יתרונות

- שיעור Positive False נמוך מאוד.
- מספק מידע מודיעיני על התוקף.
- מאפשר גילוי מוקדם של מתקפות.

חסרונות

- מזהה רק תוקפים שבאו במגע עם המלכודת.
- דורש תחזוקה ועדכון.
- תוקפים מנוסים עשויים לזהות את המלכודת.

מלכודות דבש ודאטה סיינס

מלכודות דבש מייצרות נתונים יקרי ערך לצורך:

- זיהוי אנומליות.
- למידת דפוסי תקיפה.
- בניית מערכות IDS.
- אימון מודלי Learning Machine.
- ניתוח גרפים של תוקפים ותשתיות תקיפה.

חשוב לזכור:

במערכת Honeypot, עצם הגישה למשאב מהווה פעמים רבות אנומליה בפני עצמה.

גניבת מודלים --- Theft Model

גניבת מודלים (Model Theft או Model Extraction) היא מתקפה שבה תוקף מנסה לשחזר או להעתיק מודל למידת מכונה שנבנה על ידי ארגון אחר. לעיתים המודל עצמו מהווה נכס יקר ערך, שכן פיתוחו דורש:

- איסוף נתונים.
- תיוג נתונים.
- משאבי חישוב.
- מומחיות מקצועית.
- זמן פיתוח רב.

לכן גניבת המודל עשויה לגרום לנזק כלכלי משמעותי.

כיצד מתבצעת גניבת מודלים?

במערכות רבות המודל נגיש דרך API. התוקף שולח קלטים רבים למערכת:

$$x_1, x_2, \dots, x_n$$

ומתעד את הפלטים:

$$f(x_1), f(x_2), \dots, f(x_n)$$

באמצעות זוגות הקלט-פלט ניתן לאמן מודל חדש שמחקה את המודל המקורי. גישה זו נקראת:

Attack Extraction Model

דוגמה

נניח שחברה מפעילה API לזיהוי ספאם. התוקף שולח מיליוני הודעות שונות ומקבל עבור כל אחת:

Spam ☐

Spam Not ☐

או אפילו:

☐ הסתברות להיות ספאם.

באמצעות מידע זה ניתן לבנות מודל חלופי הדומה מאוד למודל המקורי.

מדוע זו בעיה?

גניבת מודלים עלולה לאפשר:

☐ גניבת קניין רוחני.

☐ עקיפת מנגנוני הגנה.

☐ פיתוח מתקפות אדוורסריות מדויקות יותר.

☐ שימוש מסחרי במודל שנגנב.

הקשר למתקפות אדוורסריות

כאשר התוקף מצליח לבנות עותק של המודל:

$$\hat{f}(x)$$

הוא יכול לבצע ניסויים על המודל המועתק במקום על המודל האמיתי. כך ניתן לפתח דוגמאות אדוורסריות ולאחר מכן להשתמש בהן נגד המערכת המקורית. תופעה זו נקראת:

Transferability

כלומר, מתקפה שפותחה על מודל אחד מצליחה לעיתים גם נגד מודל אחר.

גניבת מודלים ופרטיות

לעיתים המודל ``זוכר" מידע מהדאטה שעליו אומן.
לכן גניבת מודל עשויה לאפשר גם מתקפות נוספות כגון:

Attack. Inference Membership □

Attack. Inversion Model □

Inference Membership

המטרה היא לבדוק האם רשומה מסוימת השתתפה בתהליך האימון.
לדוגמה:

האם אדם מסוים הופיע במאגר רפואי רגיש?

Inversion Model

המטרה היא לשחזר מידע מתוך הדאטה שעליו אומן המודל.
לדוגמה:

□ תמונות פנים.

□ נתונים רפואיים.

□ מידע פיננסי.

הגנות אפשריות

□ הגבלת מספר קריאות ל-API.

□ ניטור דפוסי שימוש חריגים.

□ הוספת רעש לתשובות.

□ החזרת תוויות בלבד במקום הסתברויות מלאות.

□ שימוש ב-Differential Privacy.

□ Watermarking של מודלים.

Watermarking

בדומה לסימן מים במסמך, ניתן להטמיע במודל התנהגות ייחודית.
אם מודל גנוב יגיב באופן זהה לקלטים מיוחדים, ניתן להוכיח בעלות על המודל.

דאטה סיינס וגניבת מודלים

מנקודת מבט של דאטה סיינס, גניבת מודלים היא בעיה המשלבת:

□ אבטחת מידע.

□ פרטיות.

□ למידת מכונה.

□ קניין רוחני.

□ ניתוח אנומליות.

לכן היא נחשבת לאחד האיומים המרכזיים על מערכות AI מודרניות.

נקודה למבחן

חשוב להבחין בין:

המטרה	התקפה
העתקת המודל	Theft Model
גילוי מי היה בדאטה	Inference Membership
שחזור מידע מהדאטה	Inversion Model
הטעיית המודל	Attack Adversarial

חישוב סטטיסטיקות מבלי לחשוף את הנתונים

לעיתים ארגון מעוניין לחשב סטטיסטיקות על קבוצת משתמשים מבלי לחשוף את הנתונים האישיים של כל אחד מהם. לדוגמה:

□ ציוני סטודנטים.

□ משכורות עובדים.

□ נתונים רפואיים.

□ נתוני שימוש במערכת.

השאלה היא כיצד ניתן לחשב סטטיסטיקות כלליות מבלי לחשוף את הערכים הפרטניים.

חישוב ממוצע

נניח שלכל אחד מ- n סטודנטים יש ציון:

$$x_1, x_2, \dots, x_n$$

הממוצע מוגדר:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

שימו לב שהממוצע תלוי רק בסכום:

$$S = \sum_{i=1}^n x_i$$

לכן אין צורך לחשוף את כל הציונים.

כל משתתף יכול לתרום רק את חלקו לחישוב הסכום הכולל, ולאחר מכן לחשב את:

$$\bar{x} = \frac{S}{n}$$

כך מתקבל הממוצע מבלי לפרסם את הנתונים האישיים.

חישוב שונות וסטיית תקן

השונות מוגדרת:

$$Var(X) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

באופן שקול:

$$Var(X) = E(X^2) - E(X)^2$$

כלומר מספיק לחשב:

$$\sum x_i$$

וגם:

$$\sum x_i^2$$

מבלי לחשוף את הציונים עצמם.
לאחר מכן:

$$Var(X) = \frac{1}{n} \sum x_i^2 - \left(\frac{1}{n} \sum x_i \right)^2$$

וסטיית התקן היא:

$$\sigma = \sqrt{Var(X)}$$

לכן גם ממוצע וגם סטיית תקן ניתנים לחישוב מתוך סכומים מצטברים בלבד.

מדוע זה עובד?

הממוצע וסטיית התקן שייכים למשפחה של סטטיסטיקות הניתנות לחישוב מתוך:

□ סכומים.

□ מומנטים.

□ סכומי חזקות.

כלומר אין צורך לדעת את הערכים הבודדים.

מדוע קשה לחשב חציון?

החציון (Median) הוא הערך שנמצא באמצע הרשימה לאחר מיון.
לדוגמה:

1, 2, 4, 5, 100

החציון הוא:

4

לעומת זאת:

1, 2, 4, 5, 6

החציון הוא:

4

בשני המקרים:

$$\sum x_i = 112$$

לעומת

$$\sum x_i = 18$$

אך גם אם נדע את הסכום ואת סכום הריבועים, עדיין לא נוכל לשחזר את מיקום הערכים. החציון תלוי בסדר הנתונים ולא רק בסכומם. לכן נדרש מידע נוסף על ההתפלגות.

מדוע קשה לחשב מקסימום?

המקסימום מוגדר:

$$\max(x_1, \dots, x_n)$$

כדי לדעת מהו הערך הגדול ביותר, יש צורך לדעת האם קיים משתתף שהחזיק ערך גבוה במיוחד. לדוגמה:

50, 50, 50, 50

לעומת:

1, 1, 1, 197

בשני המקרים:

$$\sum x_i = 200$$

אך המקסימום שונה לחלוטין. לכן לא ניתן לחשב מקסימום מתוך הסכום בלבד.

הקשר לפרטיות

היכולת לחשב:

□ ממוצע.

□ שונות.

□ סטיית תקן.

□ מומנטים.

מבלי לחשוף נתונים בודדים היא הבסיס לטכניקות רבות כגון:

□ (SMC). Computation Multiparty Secure

□ Learning. Federated

□ Privacy. Differential

□ Aggregation. Secure

לעומת זאת, חציון, אחוזונים, מינימום ומקסימום דורשים בדרך כלל מידע נוסף על התפלגות הנתונים, ולכן קשה יותר לחשב אותם תוך שמירה מלאה על פרטיות.

בעיית הסגמנט הקטן והחלקת לפלס

מהי בעיית הסגמנט הקטן?

בפרויקטי דאטה סיינס וסייבר נהוג לחלק את האוכלוסייה לסגמנטים (Segments). לדוגמה:

□ מדינה.

□ סוג מכשיר.

□ מערכת הפעלה.

□ תפקיד בארגון.

□ קבוצת גיל.

לאחר החלוקה מחשבים סטטיסטיקות שונות עבור כל סגמנט. הבעיה מתחילה כאשר אחד הסגמנטים קטן מאוד. לדוגמה:

סגמנט	מספר משתמשים
Windows	10,000
Linux	8,000
Mac	4,000
TempleOS	2

הסגמנט האחרון מכיל מעט מאוד תצפיות.

מדוע זו בעיה?

כאשר מספר הדוגמאות קטן:

□ האומדן בעל שונות גבוהה.

□ כל תצפית משפיעה בצורה קיצונית.

□ המסקנות אינן יציבות.

□ קל לבצע Overfitting.

לדוגמה:
 בסגמנט מסוים יש רק משתמש אחד.
 אותו משתמש נפרץ.
 נקבל:

$$P(\text{Attack}) = \frac{1}{1} = 100\%$$

למרות שאין מספיק מידע כדי להסיק שהסגמנט באמת מסוכן.

דוגמה בסייבר

נניח שאנו רוצים להעריך:

$$P(\text{Malware} \mid \text{System Operating})$$

ומתקבל:

Malware	Users	OS
50	1000	Windows
10	500	Linux
1	1	TempleOS

ללא תיקון נקבל:

$$P(\text{Malware} \mid \text{TempleOS}) = 1$$

כלומר:

$$100\%$$

זוהי מסקנה בעייתית מאוד.

Smoothing Laplace

הרעיון של החלקת לפלס הוא להוסיף תצפיות וירטואליות.
 במקום לחשב:

$$P(A) = \frac{\text{Successes}}{\text{Observations}}$$

מחשבים:

$$P(A) = \frac{\text{Successes} + 1}{\text{Observations} + K}$$

כאשר:

K הוא מספר הקטגוריות האפשריות.

K (1 הוא פסאודו-ספירה (Pseudo Count).

דוגמה

נניח שיש שתי אפשרויות:

$Attack$

-1

$NoAttack$

לכן:

$$K = 2$$

עבור TempleOS:

$$P(Attack) = \frac{1+1}{1+2} = \frac{2}{3}$$

במקום:

1

ההסתברות עדיין גבוהה, אך פחות קיצונית.

אינטואיציה

לפני שראינו את הנתונים אנו מניחים שכל האפשרויות קיימות.
לכן איננו מאפשרים להסתברות להגיע מיד ל:

0

או

1

רק בגלל מספר קטן של תצפיות.

דוגמה קלאסית --- Bayes Naive

נניח שבמערכת זיהוי ספאם מעולם לא הופיעה המילה:

$bitcoin$

במיילים תקינים.

ללא Laplace נקבל:

$$P(bitcoin | Legit) = 0$$

כתוצאה מכך:

$$P(Legit | Email) = 0$$

כלומר מייל אחד עלול להפוך את כל המכפלה לאפס.
החלקת לפלס מונעת בעיה זו.

יתרונות

- יציבות טובה יותר.
- התמודדות עם סגמנטים קטנים.
- מניעת הסתברויות אפס.
- הקטנת Overfitting.

חסרונות

- מכניסה הטיה מסוימת.
- עשויה להחליק יותר מדי כאשר הדאטה קטן מאוד.

נקודה חשובה למבחן

בעיית הסגמנט הקטן נובעת מכך שמספר תצפיות קטן יוצר אומדנים לא יציבים. Smoothing Laplace פותרת זאת באמצעות הוספת מידע מוקדם (Prior) בצורה של פסאודו-ספירות, ובכך מונעת הסתברויות קיצוניות ומקטינה את הסיכון להסקת מסקנות שגויות.

השוואת סגמנטים באמצעות מטריקות מרחק

לעיתים קרובות בדאטה סיינס ובסייבר אנו מעוניינים להשוות בין סגמנטים שונים. לדוגמה:

- משתמשים ממדינות שונות.
- לקוחות מסוגים שונים.
- תחנות עבודה מול שרתים.
- משתמשים תקינים מול משתמשים חשודים.
- פעילות לפני ואחרי תקיפה.

השאלה היא:

עד כמה שתי ההתפלגויות שונות זו מזו?

לצורך כך משתמשים במטריקות מרחק בין התפלגויות.

השוואת קבוצות --- T-Test ו-Mann-Whitney

לעיתים אנו מעוניינים לבדוק האם קיימים הבדלים מובהקים בין שתי קבוצות. לדוגמה: האם זמן התגובה של מחשבים נגועים שונה מזמן התגובה של מחשבים תקינים, או האם משתמשים שעברו מתקפת פשינג מתנהגים אחרת ממשתמשים שלא נפגעו. כאשר הנתונים רציפים ומקיימים בקירוב הנחת נורמליות, ניתן להשתמש ב-T-Test להשוואת הממוצעים בין שתי הקבוצות. לעומת זאת, כאשר הנתונים אינם נורמליים, מכילים חריגים רבים או מבוססים על דירוגים, נהוג להשתמש במבחן Mann-Whitney U, שהוא מבחן לא פרמטרי המבוסס על דירוג הנתונים. שני המבחנים בודקים את השערת האפס שלפיה אין הבדל בין הקבוצות, ומחזירים ערך p -value המאפשר להעריך האם ההבדל שנצפה

עשוי להיות מוסבר על ידי מקריות בלבד. בעולם הסייבר מבחנים אלו משמשים בין היתר להשוואת סגמנטים של משתמשים, זיהוי שינויי התנהגות, בדיקת השפעת מתקפות, והשוואת ביצועים של מנגנוני הגנה שונים.

(EMD) Distance Mover Earth

נקראת גם:

Wasserstein Distance

הרעיון:

נניח ששתי ההתפלגויות הן ערימות חול. EMD מודדת כמה עבודה יש לבצע כדי להעביר את החול מערימה אחת לאחרת. באופן אינטואיטיבי:

$$Distance = Amount \times Movement$$

יתרונות:

□ אינטואיטיבית.

□ רגישה למרחק בין הערכים.

□ מתאימה למשתנים רציפים.

דוגמה:

$$P = (10, 0, 0)$$

$$Q = (0, 0, 10)$$

המרחק גדול משום שיש להזיז את כל המסה מקצה אחד לקצה השני.

שימושים בסייבר

□ השוואת התפלגות גודל חבילות.

□ השוואת זמני התחברות.

□ גילוי שינוי התנהגות לאורך זמן.

□ Detection, Drift

Divergence KL

המרחק של קולבק-לייבלר:

$$D_{KL}(P||Q) = \sum_i P(i) \log \left(\frac{P(i)}{Q(i)} \right)$$

הרעיון:

מודד כמה מידע הולך לאיבוד כאשר משתמשים ב-Q כדי לתאר את P. תכונות:

- אינו סימטרי.
- אינו מרחק אמיתי.
- נפוץ מאוד בתורת האינפורמציה.
- באופן כללי:

$$D_{KL}(P||Q) \neq D_{KL}(Q||P)$$

שימושים בסייבר

- זיהוי שינוי בהתנהגות משתמשים.
- גילוי מתקפות חדשות.
- השוואת התפלגות פקודות.
- זיהוי Drift, Data

Divergence Jensen-Shannon

גרסה סימטרית של KL.
מוגדרת:

$$JS(P, Q) = \frac{1}{2} D_{KL}(P||M) + \frac{1}{2} D_{KL}(Q||M)$$

כאשר:

$$M = \frac{P + Q}{2}$$

יתרונות:

- סימטרית.
- תמיד חסומה.
- יציבה יותר מ-KL.

(KS) Statistic Kolmogorov-Smirnov

מטריקה המשווה בין שתי פונקציות התפלגות מצטברות.
מוגדרת:

$$KS = \sup_x |F_1(x) - F_2(x)|$$

כלומר:

הפער המקסימלי בין שתי עקומות ההתפלגות המצטברות.
יתרונות:

- אינו מניח התפלגות מסוימת.
- קל לחישוב.
- נפוץ מאוד בזיהוי Drift.

שימושים בסייבר

- השוואת התנהגות משתמשים.
- השוואת תעבורת רשת בין תקופות.
- גילוי שינוי במאפייני תקיפה.

Distance Variation Total

מוגדר:

$$TV(P, Q) = \frac{1}{2} \sum_i |P(i) - Q(i)|$$

הרעיון:
מודד כמה מסה צריך להעביר כדי להפוך התפלגות אחת לאחרת.
ערכים:

$$0 \leq TV \leq 1$$

Distance Hellinger

מוגדר:

$$H(P, Q) = \frac{1}{\sqrt{2}} \sqrt{\sum_i (\sqrt{P_i} - \sqrt{Q_i})^2}$$

יתרונות:

- חסום בין 0 ל-1.
- סימטרי.
- יציב יחסית.

מתי להשתמש במה?

שימוש אופייני	מטריקה
השוואת התפלגויות רציפות	Wasserstein / EMD
תורת אינפורמציה, Drift	Divergence KL
השוואה סימטרית בין התפלגויות	Jensen-Shannon
השוואת התפלגויות ללא הנחות	KS
הבדל הסתברותי כללי	Variation Total
השוואת התפלגויות הסתברותיות	Hellinger

דוגמה בסייבר

נניח שבחודש ינואר התפלגות שעות הכניסה למערכת הייתה:

$$P$$

ובחודש פברואר:

$$Q$$

אם:

$$KS(P, Q)$$

או

$$EMD(P, Q)$$

גבוהים במיוחד,
ייתכן שהתרחש:

□ שינוי התנהגות משתמשים.

□ מתקפה.

□ שינוי ארגוני.

□ תקלה במערכת.

נקודה למבחן

כל המטריקות הללו מנסות לענות על אותה שאלה:

עד כמה שני סגמנטים שונים זה מזה?

ההבדל הוא בדרך שבה כל מטריקה מודדת את מושג ה"שונות" בין ההתפלגויות.

id="graphcommunity1" `` ` latex

שאלה למחשבה: כאשר אתם מייצרים דטה סינטטי איך הייתם משתמשים במטריקות הללו על מנת לוודא שהדטה שייצרתם הוא איכותי אבל לא פוגע בפרטיות

מציאת קהילות בגרפים

אחת הבעיות המרכזיות בניתוח גרפים היא זיהוי קהילות (Community Detection). קהילה היא קבוצת קודקודים המקיימת:

□ קשרים רבים בתוך הקבוצה.

□ מעט קשרים מחוץ לקבוצה.

בסייבר ניתן לראות קהילות כ:

□ קבוצות מחשבים המתקשרים ביניהם.

□ קבוצות משתמשים בעלי פעילות דומה.

□ בוטנטים.

□ קבוצות שרתים בארגון.

□ אזורים נגועים ברשת.

קליקות (Cliques)

הגדרה:

קליקה היא קבוצה של קודקודים שבה כל זוג קודקודים מחובר.

לדוגמה:

אם קיימים הקודקודים:

$$A, B, C, D$$

וכל אחד מחובר לכל השאר,

נקבל קליקה בגודל 4.

מספר הקשתות יהיה:

$$\frac{n(n-1)}{2}$$

מדוע קליקה נראית כמו קהילה מושלמת?

בתוך קליקה:

□ הצפיפות היא 100%.

□ כל חבר מכיר את כל האחרים.

□ אין קהילה צפופה יותר מקליקה.

לכן באופן אינטואיטיבי נראה הגיוני לחפש קליקות.

מדוע זה אינו פתרון מעשי?

מציאת הקליקה המקסימלית (Maximum Clique)

היא בעיה NP-Hard.

זמן הריצה גדל בצורה אקספוננציאלית:

$$O(2^n)$$

בקירוב.

בגרפים אמיתיים המכילים:

□ אלפי משתמשים.

□ מיליוני מחשבים.

□ מיליארדי חיבורים.

החישוב אינו מעשי.
בנוסף:
קהילות אמיתיות אינן קליקות מושלמות.
לכן מחפשים הגדרות רכות יותר.

מודולריות (Modularity)

מודולריות היא אחד המדדים הנפוצים ביותר לזיהוי קהילות.
הרעיון:
נשווה בין:

□ מספר הקשתות בתוך הקהילה.

□ מספר הקשתות שהיה צפוי באופן אקראי.

הנוסחה:

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

כאשר:

□ A_{ij} היא מטריצת השכנויות.

□ k_i היא דרגת הקודקוד.

□ m מספר הקשתות.

□ δ שווה 1 אם שני הקודקודים באותה קהילה.

באופן אינטואיטיבי:

$$\text{Modularity} = \text{Observed} - \text{Expected}$$

ככל שהמודולריות גדולה יותר,
הקהילות ברורות יותר.

אלגוריתם Louvain

אלגוריתם Louvain מנסה למקסם את המודולריות.
יתרונות:

□ מהיר מאוד.

□ מתאים לגרפים גדולים.

□ נפוץ בסייבר.

Laplacian של גרף

מטריצת הלפלסיאן מוגדרת:

$$L = D - A$$

כאשר:

A מטריצת השכנויות.

D מטריצת הדרגות.

מדוע הלפלסיאן חשוב?

הלפלסיאן מתאר את מבנה הגרף. הערכים העצמיים והווקטורים העצמיים שלו מכילים מידע רב על:

□ קישוריות.

□ קהילות.

□ צווארי בקבוק.

□ זרימת מידע.

הערך העצמי השני

נסמן:

$$0 = \lambda_1 \leq \lambda_2 \leq \lambda_3 \leq \dots$$

הערך:

$$\lambda_2$$

נקרא:

Value Fiedler

והווקטור המתאים:

$$v_2$$

נקרא:

Vector Fiedler

באמצעות סימן הרכיבים של v_2

ניתן לחלק את הגרף לשתי קהילות.

חשוב לזכור. הלפלסיאן הוא הרחבה במבנה הדיפרציאלי לגרפים. שימו לב למה שקורה כאשר הגרף השריג. כאשר הלפלסיאן מאפשר לנסח את בעיית החוס על גרפים. מעבר החוס שקול למעבר המידע

Clustering Spectral

רעיון מרכזי:

1. לחשב את הלפלסיאן.
2. לחשב את הווקטורים העצמיים.
3. להטיל את הקודקודים למרחב חדש.
4. להפעיל K-Means.

זהו אחד הכלים החזקים ביותר למציאת קהילות.

חתך מינימלי (Minimum Cut)

נרצה לחלק את הגרף לשני חלקים.
מחיר החתך:

$$Cut(S, T)$$

הוא מספר הקשתות שיש להסיר.
המטרה:

$$\min Cut(S, T)$$

כלומר למצוא את החלוקה הזולה ביותר.

בעיה

לעיתים החתך המינימלי ינתק קודקוד בודד.
לכן משתמשים בגרסאות משופרות כגון:

Cut. Normalized $\square$

Cut. Ratio $\square$

משפט הזרימה המקסימלית והחתך המינימלי

אחד המשפטים היפים במדעי המחשב.
נתון גרף עם:

$\square$ מקור s

$\square$ יעד t

המשפט אומר:

$$\text{Flow Maximum} = \text{Cut Minimum}$$

כלומר:

הזרימה המקסימלית שניתן להעביר ברשת
שווה לקיבולת החתך המינימלי.

אינטואיציה

אם רוצים לנתק את התקשורת בין:

s

לבין

t

מספיק להסיר את הקשתות של החתך המינימלי.

יישומים בסייבר

□ זיהוי אזורים נגועים.

□ בידוד מתקפה.

□ זיהוי בוטנטים.

□ זיהוי מחשבים קריטיים.

□ תכנון. Micro-Segmentation.

□ ניתוח תעבורת רשת.

סיכום

שיטה	רעיון מרכזי
Clique	קהילה מושלמת אך NP-Hard
Modularity	יותר קשתות מהמצופה אקראית
Laplacian	שימוש בערכים עצמיים
Clustering Spectral	קלאסטרינג על וקטורים עצמיים
Cut Minimum	חיתוך מינימלי של הגרף
Cut Min = Flow Max	קשר בין זרימה לחתך

נקודה למבחן:

קהילה בגרף אינה בהכרח קליקה.

בפועל מחפשים קבוצות עם הרבה קשרים פנימיים ומעט קשרים חיצוניים, ולכן משתמשים במודולריות, לפלסיאן, Clustering Spectral וחתיכים מינימליים במקום חיפוש קליקות.

גרפים של רשתות חברתיות וסייבר

רשת חברתית ניתנת לייצוג באמצעות גרף.

בגרף זה:

□ כל משתמש מיוצג על ידי קודקוד (Vertex).

□ קשר בין משתמשים מיוצג על ידי קשת (Edge).

דוגמאות:

Facebook □

LinkedIn □

Instagram □

(Twitter) X □

□ רשת תקשורת ארגונית

□ רשת מחשבים

מפניע לגלות שרבות מהתכונות המתמטיות של רשתות חברתיות מופיעות גם ברשתות מחשבים, רשתות תקשורת ורשתות תקיפה.

מדדי מרכזיות (Centrality Measures)

המטרה:

לזהות קודקודים חשובים בגרף.

Centrality Degree

מדד המרכזיות הפשוט ביותר.

$$\text{Degree}(v)$$

הוא מספר השכנים של הקודקוד.

לדוגמה:

אם למשתמש יש:

$$200$$

חברים,

אז:

$$\text{Degree} = 200$$

Centrality Betweenness

מודד כמה מסלולים קצרים עוברים דרך הקודקוד.

$$BC(v) = \sum_{s,t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

כאשר:

□ σ_{st} הוא מספר המסלולים הקצרים בין s ל- t .

□ $\sigma_{st}(v)$ הוא מספר המסלולים העוברים דרך v .

אינטואיציה:

זהו ``צומת מעבר" חשוב.

Centrality Closeness

מודד כמה מהר ניתן להגיע מכל קודקוד לאחרים.

$$CC(v) = \frac{1}{\sum_u d(v, u)}$$

קודקודים מרכזיים יקבלו ערך גבוה.

Centrality Eigenvector

לא כל חבר שווה.

חיבור לאנשים חשובים מגדיל את החשיבות שלך.

$$Ax = \lambda x$$

זהו אותו רעיון שהוביל ליצירת PageRank.

התפלגות חזקה (Heavy-Tailed Distribution)

ברשתות חברתיות ההתפלגות של מספר החברים אינה נורמלית. במקום זאת מתקבלת לעיתים קרובות:

$$P(k) \propto k^{-\alpha}$$

התפלגות מסוג Law. Power

מה משמעות הדבר?

רוב האנשים מחזיקים מעט חברים.
מעט אנשים מחזיקים מספר עצום של חברים.
לדוגמה:

□ רוב המשתמשים: 50--300 חברים.

□ משפיענים: מיליוני עוקבים.

מדוע זה קורה?

אחד ההסברים הוא:

Attachment Preferential

העשיר מתעשר.

אם למישהו יש כבר הרבה חברים,
קל יותר לצבור חברים נוספים.
לכן:

□ פופולריים הופכים לפופולריים יותר.

□ נוצרים ים-Hub

פרדוקס החברים

אחת התוצאות המפתיעות ביותר בתורת הרשתות.
עבור רוב האנשים:

לחברים שלך יש בממוצע יותר חברים מאשר לך.

למה זה קורה?

נניח שיש:

□ הרבה משתמשים עם מעט חברים.

□ מעט משתמשים עם המון חברים.

הסיכוי לפגוש אדם בעל הרבה חברים גבוה יותר,
משום שהוא מופיע כשכן של הרבה אנשים.
במילים אחרות:
קודקודים בעלי דרגה גבוהה ``נספרים'' פעמים רבות.

דוגמה

נניח:

100

משתמשים רגילים בעלי:

10

חברים.

ומשתמש אחד בעל:

1000

חברים.

כאשר בוחרים חבר אקראי של משתמש,
קיימת הסתברות גבוהה להגיע דווקא לאותו Hub.
לכן ממוצע החברים של החברים גדול ממספר החברים של רוב המשתמשים.

ניסוח מתמטי

נסמן:

k

כמספר השכנים.
הממוצע הרגיל:

$E[k]$

לעומת ממוצע השכנים של השכנים:

$$\frac{E[k^2]}{E[k]}$$

מאחר ש:

$$E[k^2] \geq E[k]^2$$

נקבל בדרך כלל:

$$\frac{E[k^2]}{E[k]} > E[k]$$

וזהו בדיוק פרדוקס החברים.

למה זה חשוב בסייבר?

זיהוי מוקדם של התפשטות

נניח שמתקפה מתפשטת ברשת.
אם נדגום:

□ משתמשים אקראיים.

נגלה את המתקפה מאוחר יחסית.
אם נדגום:

□ חברים של משתמשים אקראיים.

נגיע באופן טבעי לקודקודים מרכזיים יותר.
לכן נזהה את המתקפה מוקדם יותר.

זיהוי בוטנטים

בוטנטים רבים בנויים סביב Hub-ים.
מדדי מרכזיות מסייעים לזהות:

□ שרתי פיקוד ושליטה.

□ מחשבים קריטיים.

□ מפיצי תקיפה.

הקשחת רשת

אם רוצים להגן על רשת:
לא תמיד צריך להגן על כולם.
לעיתים מספיק להגן על:

□ קודקודים בעלי Degree גבוה.

□ קודקודים בעלי Betweenness גבוה.

כדי להקטין משמעותית את הסיכון להתפשטות.

ניתוח מתקפות

בגרף תקשורת ארגוני ניתן לזהות:

- מי ניגש למי.
- אילו שרתים קריטיים.
- אילו מחשבים משמשים כגשר בין מחלקות.
- כיצד תוקף נע ברשת.

נקודות למבחן

1. Centrality Degree = מספר השכנים.
2. Centrality Betweenness = כמה מסלולים עוברים דרכך.
3. Centrality Closeness = כמה מהר ניתן להגיע לאחרים.
4. Centrality Eigenvector = חשוב אם החברים שלך חשובים.
5. רשתות חברתיות מקיימות לעיתים קרובות Law. Power.
6. רוב האנשים מחזיקים פחות חברים מהממוצע של החברים שלהם.
7. הסיבה היא שקודקודים בעלי דרגה גבוהה מופיעים כשכנים של הרבה אנשים.
8. תכונה זו מאפשרת גילוי מוקדם של התפשטות מתקפות וזיהוי ים-Hub קריטיים ברשת.

Skewness --- אסימטריה בהתפלגות

אחת השאלות הראשונות שיש לשאול בעת ניתוח נתונים היא:

האם ההתפלגות סימטרית?

מדד האסימטריה (Skewness) מודד עד כמה ההתפלגות מוטה לצד אחד. כאשר ההתפלגות סימטרית:

$$Mean \approx Median \approx Mode$$

ככל שההתפלגות פחות סימטרית, הפער בין מדדים אלו גדל.

סוגי אסימטריה

אסימטריה חיובית (Right Skew)

הזנב הארוך נמצא בצד ימין.
דוגמאות:

- משכורות.
- עושר.

□ מספר חברים ברשת חברתית.

□ מספר הורדות קבצים.

□ מספר ניסיונות תקיפה.

בדרך כלל:

$$Mode < Median < Mean$$

אסימטריה שלילית (Left Skew)

הזנב הארוך נמצא בצד שמאל.
דוגמאות:

□ ציונים במבחן קל במיוחד.

□ זמני תגובה במערכת יעילה.

בדרך כלל:

$$Mean < Median < Mode$$

מדד הסקיו המתמטי

ההגדרה הקלאסית:

$$Skewness = \frac{E[(X - \mu)^3]}{\sigma^3}$$

כאשר:

□ μ הוא הממוצע.

□ σ היא סטיית התקן.

פירוש:

□ $Skewness = 0$: סימטריה.

□ $Skewness > 0$: זנב ימני.

□ $Skewness < 0$: זנב שמאלי.

שלושת מדדי הסקיו שלמדנו

Coefficient Skewness First Pearson

$$Sk_1 = \frac{Mean - Mode}{\sigma}$$

יתרון:

□ אינטואיטיבי.

חסרון:

□ דורש Mode יציב.

Coefficient Skewness Second Pearson

$$Sk_2 = 3 \frac{\text{Mean} - \text{Median}}{\sigma}$$

נפוץ יותר בפועל.
אינו דורש חישוב Mode.

Skewness Bowley

מבוסס רבעונים.

$$Sk_B = \frac{(Q_3 + Q_1) - 2Q_2}{Q_3 - Q_1}$$

כאשר:

$$Q_2 = \text{Median}$$

יתרונות:

- עמיד יחסית לחריגים.
- מתאים לנתונים בעלי זנבות כבדים.

מדוע Skewness חשוב בסטטיסטיקה?

התפלגויות רבות בסטטיסטיקה מניחות:

- סימטריה.
- נורמליות.
- שונות יציבה.

כאשר קיימת אסימטריה חזקה:

- הממוצע עלול להיות מטעה.
- סטיית התקן פחות אינפורמטיבית.
- מבחנים סטטיסטיים עלולים להיפגע.
- רווחי סמך עלולים להיות לא מדויקים.

מדוע Skewness חשוב בסייבר?

רבים מהמשתנים בסייבר הם בעלי אסימטריה חזקה. לדוגמה:

□ מספר כניסות למערכת.

□ מספר הורדות.

□ גודל קבצים.

□ מספר חיבורים.

□ זמן בין מתקפות.

לרוב:

□ רוב המשתמשים כמעט אינם פעילים.

□ מעט משתמשים מייצרים פעילות עצומה.

לכן מתקבלות התפלגויות בעלות זנב ימני ארוך.

השפעה על אנומליות

כאשר ההתפלגות מוטה:

□ Z-Score עלול לזהות יותר מדי חריגים.

□ הממוצע עלול להתרחק ממרכז הנתונים.

□ השונות עלולה להיות גדולה מאוד.

לכן לעיתים עדיף להשתמש ב:

Median □

MAD □

IQR □

במקום Mean ו-Standard Deviation.

כיצד מתקנים אסימטריה?

Transform Log

הטרנספורמציה הנפוצה ביותר.

$$Y = \log(X)$$

מתאימה כאשר:

□ כל הערכים חיוביים.

□ קיימת אסימטריה ימנית חזקה.

דוגמה:

1, 10, 100, 1000

יהפוך ל:

0, 1, 2, 3

בקירוב.

Transform Root Square

$$Y = \sqrt{X}$$

טרנספורמציה מתונה יותר.
נפוצה כאשר מדובר בספירות.
לדוגמה:

□ מספר אירועים.

□ מספר מתקפות.

Transform Root Cube

$$Y = \sqrt[3]{X}$$

מתאימה גם לערכים שליליים.

Transformation Box-Cox

משפחה כללית של טרנספורמציות.

$$Y = \frac{X^\lambda - 1}{\lambda}$$

כאשר:

$$\lambda$$

נבחר מתוך הנתונים.
מקרים מיוחדים:

$$\lambda = 1$$

ללא שינוי.

$$\lambda = 0$$

שקול ל-Log Transform.
יתרון:

מאפשר למצוא אוטומטית טרנספורמציה מתאימה.

Transformation Yeo-Johnson

דומה ל-Box-Cox.

יתרון:

מתאימה גם כאשר קיימים ערכים שליליים.

נקודות למבחן

1. Skewness מודד אסימטריה בהתפלגות.

2. Skew Right נפוץ מאוד בסייבר.

3. במצב כזה לרוב:

$$Mode < Median < Mean$$

4. שלושת המדדים שלמדנו:

□ Pearson ראשון.

□ Pearson שני.

□ Bowley.

5. אסימטריה חזקה עלולה לפגוע במודלים ובמבחנים סטטיסטיים.

6. Transform Log היא הטרנספורמציה הנפוצה ביותר.

7. Box-Cox ו-Yeo-Johnson הן טרנספורמציות כלליות יותר.

8. במקרים רבים Median ו-IQR יציבים יותר מממוצע וסטיית תקן.

משתני זמן וחשיבותם בניתוח נתונים

משתני זמן הם מהמשתנים החשובים ביותר בדאטה סיינס ובסייבר. פעמים רבות התנהגות שנראית תקינה לחלוטין כאשר בוחנים אירועים בנפרד הופכת לחשודה כאשר מוסיפים את ממד הזמן. דוגמאות למשתני זמן:

- תאריך.
- שעה.
- יום בשבוע.
- חודש.
- עונה.
- משך זמן בין אירועים.
- זמן מאז הפעולה האחרונה.

אזורי זמן (Time Zones)

בעת עבודה עם נתונים גלובליים חשוב להתייחס לאזור הזמן שבו התרחש האירוע. לדוגמה:

- משתמש בישראל מתחבר בשעה 09:00.
- משתמש בארצות הברית מתחבר בשעה 09:00.
- למרות שהשעה זהה, מדובר בזמנים שונים לחלוטין. לכן נהוג לשמור בנוסף לשעה המקומית גם:

UTC

שהוא הזמן האוניברסלי. אי התחשבות באזורי זמן עלולה לגרום לטעויות בניתוח סדרות זמן, זיהוי אנומליות ובניית מודלים.

שעון קיץ (Daylight Saving Time)

במדינות רבות השעה משתנה פעמיים בשנה. במעבר לשעון קיץ:

- שעה אחת נעלמת.
 - במעבר לשעון חורף:
 - שעה אחת מופיעה פעמיים.
- לדוגמה:

02:30

עשויה להופיע פעמיים באותו יום. אם לא מטפלים נכון במעבר זה עלולות להיווצר:

□ כפילויות.

□ פערי זמן שגויים.

□ חישובי משך שגויים.

□ טעויות בניתוח סדרות זמן.

לכן במערכות רבות נהוג לשמור את הזמן ב-UTC ולהמיר לשעון מקומי רק לצורכי תצוגה.

משתני זמן מחזוריים

אחת הטעויות הנפוצות היא להתייחס לשעה כאל משתנה רגיל.
לדוגמה:

23 : 59

-1

00 : 01

קרובות מאוד זו לזו.
אך מספרית:

23.98

-1

0.02

נראות רחוקות.
לכן לעיתים מקודדים זמן באמצעות:

$$\sin\left(\frac{2\pi t}{T}\right)$$

-1

$$\cos\left(\frac{2\pi t}{T}\right)$$

כאשר T הוא אורך המחזור.
לדוגמה:

□ 24 שעות ביממה.

□ 7 ימים בשבוע.

□ 12 חודשים בשנה.

שיטה זו שומרת על המבנה המעגלי של הזמן.

שימושים בסייבר

למשתני זמן יש חשיבות רבה בזיהוי מתקפות:

- כניסות בשעות חריגות.
- גידול פתאומי במספר אירועים.
- פעילות מחוץ לשעות העבודה.
- שינוי בדפוסי עבודה רגילים.
- זיהוי מתקפות מתמשכות.
- זיהוי מתקפות מתוזמנות.

לדוגמה:

עובד המתחבר בכל יום בין 08:00 ל-17:00 ומבצע לפתע פעילות ענפה בשעה 03:00 עשוי להוות אינדיקציה לחשבון שנפרץ.

נקודות למבחן

- יש להתייחס תמיד לאזור הזמן שבו נאסף המידע.
- מומלץ לשמור זמנים בפורמט UTC.
- יש לשים לב למעברי שעון קיץ וחורף.
- שעה, יום בשבוע וחודש הם משתנים מחזוריים.
- זמן הוא אחד המשתנים החשובים ביותר בזיהוי אנומליות בסייבר.

הקשר הזמן --- יותר מסתם תאריך ושעה

בעת ניתוח נתונים חשוב להבין שזמן אינו רק מספר. אירועים רבים מושפעים מהקשר חיצוני שאינו מופיע ישירות בנתונים. לכן פעמים רבות נדרש לבצע Feature Engineering על משתני הזמן.

אירועים מיוחדים

התנהגות של מערכות, משתמשים וארגונים עשויה להשתנות באופן משמעותי בתקופות מסוימות. דוגמאות:

- חורף לעומת קיץ.
- יום לעומת לילה.
- ימי חול לעומת סופי שבוע.
- חגים.
- חופשות.
- מלחמות.

□ מגפות כגון COVID-19.

□ אירועי ספורט גדולים.

□ בחירות.

□ השקות מוצרים.

לכן לעיתים כדאי ליצור משתנים חדשים כגון:

IsHoliday □

IsWeekend □

IsWarPeriod □

IsCovidPeriod □

IsWorldCup □

IsNight □

דוגמאות מסייבר

יום מול לילה

בארגונים רבים רוב העובדים פעילים בשעות היום.
לכן:

□ התחברות ב-10:00 בבוקר עשויה להיות תקינה.

□ התחברות ב-03:00 בלילה עשויה להיות חריגה.

חגים

במהלך חגים:

□ נפח הפעילות משתנה.

□ צוותי אבטחה קטנים יותר.

□ תוקפים מנצלים את הירידה בערנות.

מלחמות

במהלך מלחמות לעיתים נצפה:

□ גידול במתקפות סייבר.

□ שינוי בדפוסי עבודה.

□ מעבר לעבודה מרחוק.

קורונה

תקופת COVID-19 שינתה לחלוטין:

□ דפוסי עבודה.

□ תעבורת רשת.

□ שימוש ב-VPN.

□ הרגלי צריכה.

מודלים שנבנו לפני התקופה לעיתים הפסיקו לעבוד היטב לאחריה.
זוהי דוגמה קלאסית ל:

Concept Drift

משחקי כדורגל

במדינות רבות משחקים גדולים משפיעים על:

□ נפח תעבורה.

□ פעילות ברשתות חברתיות.

□ היקף קניות.

□ צפייה בשירותי סטרימינג.

לכן משחק גמר עשוי ליצור התנהגות חריגה לחלוטין גם ללא מתקפה.

מדוע 1 בינואר 1970 חשוב?

במערכות רבות הזמן מיוצג באמצעות:

Unix Timestamp

שהוא מספר השניות שעברו מאז:

1 בינואר 1970
00:00:00
UTC

רגע זה נקרא:

Unix Epoch

לדוגמה:

0

מייצג את:

1970 – 01 – 01 00:00:00

UTC.

מדוע זה חשוב?

באגים רבים גורמים לשדה זמן לקבל:

0

ולכן מתקבל תאריך:

1970 – 01 – 01

כאשר רואים תאריך זה בדאטה יש לחשוד ב:

□ ערך חסר.

□ שגיאת המרה.

□ תקלה במערכת.

□ תקלה בשעון.

לכן זהו אחד התאריכים המפורסמים ביותר בעולם הדאטה.

כאשר יש יותר ממשתנה זמן אחד

במערכות רבות קיימים מספר שדות זמן.
לדוגמה:

□ זמן יצירת קובץ.

□ זמן שינוי אחרון.

□ זמן גישה אחרון.

□ זמן התחברות.

□ זמן התנתקות.

□ זמן זיהוי התקיפה.

במקרים כאלה לעיתים המידע החשוב ביותר אינו הזמן עצמו אלא:

Δt

כלומר ההפרש בין הזמנים.
לדוגמה:

$$Duration = LogoutTime - LoginTime$$

או:

$$DetectionDelay = DetectionTime - AttackTime$$

Engineering Feature על מספר זמנים

ניתן ליצור:

- משך התחברות.
 - זמן מאז הפעילות האחרונה.
 - זמן בין שתי פעולות.
 - מספר אירועים בשעה.
 - מספר אירועים ביום.
 - זמן תגובה ממוצע.
- לעיתים משתנים אלו חשובים יותר מהזמן המקורי.

נקודה למבחן

בעת ניתוח משתני זמן חשוב לשאול:

1. האם קיימים אזורי זמן שונים?
 2. האם מדובר בשעון קיץ או חורף?
 3. האם האירוע התרחש בחג או בסוף שבוע?
 4. האם קיימים אירועים חיצוניים המשפיעים על הנתונים?
 5. האם ניתן ליצור משתני זמן חדשים?
 6. האם קיימים מספר שדות זמן ומה ניתן ללמוד מהפערים ביניהם?
- לעיתים קרובות התובנות החשובות ביותר בדאטה אינן נמצאות בערך הזמן עצמו, אלא בהקשר שבו הוא התרחש וביחסים בין מספר משתני זמן.

מדוע הקורונה הקשתה על זיהוי מתקפות סייבר?

בחודש מרץ 2020 העולם נכנס לסגרים נרחבים בעקבות מגפת COVID-19. ארגונים רבים עברו לעבודה מרחוק בתוך ימים ספורים, דבר שגרם לשינוי דרמטי בדפוסי ההתנהגות של המשתמשים. במקביל נרשמה עלייה משמעותית במתקפות פשינג, מתקפות כופרה וניסיונות חדירה שניצלו את הפחד וחוסר הוודאות סביב המגפה. [contentReference\[oaicite:0\]index=0](#): הבעיה הייתה שמרבית מערכות הזיהוי התבססו על הנחה שהתנהגות העבר מייצגת את ההתנהגות העתידית. אולם במהלך מרץ 2020 התרחש שינוי קיצוני בהתנהגות הלגיטימית של המשתמשים:

- התחברויות רבות מהבית במקום מהמשרד.
- שימוש מוגבר ב-VPN.
- גידול חד בשיחות וידאו.
- שעות עבודה חריגות.

□ גישה למערכות ממדינות ואזורים חדשים.

כתוצאה מכך נוצר מצב שבו פעילות לגיטימית נראתה חריגה, ואילו חלק מהפעילות הזדונית נטמעה בתוך השינוי הכללי. תופעה זו נקראת:

Concept Drift

כלומר שינוי בהתפלגות הנתונים שעליה אומן המודל. מבחינה סטטיסטית, קו הבסיס (Baseline) השתנה בבת אחת. לכן מערכות אנומליה רבות סבלו מעלייה ב-Positives False או לחלופין הקלו את ספי הזיהוי וסיכנו את הארגון בעלייה ב-Negatives. False. זהו אחד המקרים המפורסמים ביותר המדגימים כיצד אירוע חיצוני שאינו קשור ישירות לסייבר יכול לגרום לשינוי בהתפלגות הנתונים ולהקשות משמעותית על זיהוי מתקפות. "מדוע שינוי התנהגות ארגונית עקב הקורונה מהווה אתגר למערכות זיהוי אנומליות?"

מתקפת SolarWinds --- אחת מתקפות שרשרת האספקה הגדולות בהיסטוריה

מתקפת SolarWinds נחשבת לאחת ממתקפות הסייבר המתוחכמות והמשמעותיות ביותר שהתגלו בעשור האחרון. המתקפה התגלתה בדצמבר 2020, אך בדיעבד התברר שהתוקפים הצליחו לחדור למערכות החברה כבר במהלך שנת 2019 ולשתול קוד זדוני בעדכון תוכנה שהופץ ללקוחות במהלך שנת 2020.

מהי SolarWinds?

SolarWinds היא חברת תוכנה המספקת כלי ניהול וניטור תשתיות IT. אחד ממוצריה המרכזיים הוא:

Orion

מערכת לניהול רשתות, שרתים, תחנות עבודה ותשתיות ארגוניות. המערכת נמצאת בדרך כלל בלב רשת הארגון ולכן מחזיקה הרשאות גבוהות במיוחד.

כיצד בוצעה המתקפה?

התוקפים הצליחו לחדור לתהליך הפיתוח של החברה. במקום לתקוף את הלקוחות ישירות, הם תקפו את:

Supply Chain

כלומר את שרשרת האספקה. הם השתילו קוד זדוני בתוך עדכון תוכנה לגיטימי. העדכון נחתם דיגיטלית על ידי SolarWinds והופץ ללקוחות באופן רגיל. לכן מבחינת הלקוחות:

□ הקובץ היה חתום.

□ מקורו היה אמין.

□ תהליך העדכון נראה תקין לחלוטין.

SUNBURST

הקוד הזדוני שהוחדר נקרא:

SUNBURST

לאחר ההתקנה הוא:

- המתין מספר ימים.
- יצר תקשורת החוצה.
- אסף מידע על הארגון.
- אפשר לתוקפים לבצע פעולות נוספות.

חשוב להבין:

לא כל מי שהתקין את העדכון הותקף בפועל. התוקפים בחרו מטרות ספציפיות מתוך כלל הארגונים שנדבקו.

מי נפגע?

המתקפה השפיעה על:

- סוכנויות ממשל בארצות הברית.
- חברות טכנולוגיה.
- חברות אבטחת מידע.
- ארגונים פיננסיים.
- חברות תעשייה.

ההערכה היא שעשרות אלפי לקוחות הורידו את העדכון הנגוע.

מדוע היה קשה לזהות את המתקפה?

המתקפה הייתה מתוככמת במיוחד.

1. העדכון הגיע ממקור אמין.
2. התעבורה נראתה לגיטימית.
3. הקוד הזדוני פעל באיטיות.
4. הוא התחזה לפעילות רגילה של המערכת.
5. לא בוצעו פעולות אגרסיביות מיד לאחר ההדבקה.

במילים אחרות:

התוקפים ניסו להשתלב בתוך הרעש הרגיל של הארגון.

קשר לדאטה סיינס

המתקפה מדגימה מספר עקרונות חשובים:

אנומליה אינה בהכרח קיצונית

רבים חושבים שמתקפה נראית כמו פעילות חריגה מאוד.
בפועל:
התקפה מוצלחת לעיתים קרובות נראית כמעט רגילה.
לכן:

□ Z-Score לבדו אינו מספיק.

□ נדרשת אנליזה רב-ממדית.

□ נדרש הקשר עסקי.

גרפים

ניתן לייצג את הארגון כגרף:

□ משתמשים.

□ מחשבים.

□ שרתים.

□ שירותים.

לאחר החדירה התוקפים ביצעו:

Lateral Movement

כלומר מעבר בין מערכות שונות.
ניתוח גרפים יכול לסייע בזיהוי התנהגות זו.

משתני זמן

התוקפים פעלו לאורך חודשים רבים.
לכן:

□ ניתוח סדרות זמן.

□ זיהוי Drift.

□ מעקב אחר שינויים איטיים.

עשויים להיות יעילים יותר מאשר חיפוש אחר אירוע בודד.

מדוע זו מתקפת Chain? Supply

במקום לתקוף:

1000

לקוחות בנפרד,
התוקפים תקפו ספק אחד.
אם נסמן:

Vendor → Customers

אז החדירה לספק אפשרה להגיע לכל הלקוחות.
זהו בדיוק הרעיון של מתקפת שרשרת אספקה.

לקחים מרכזיים

- אין לסמוך באופן עיוור על תוכנה חתומה.
- גם ספקים אמינים עלולים להיפרץ.
- יש לנטר שינויים איטיים ולא רק אירועים חריגים.
- יש לשלב מודלים סטטיסטיים עם ידע סייברי.
- אבטחת שרשרת האספקה היא חלק מרכזי מהגנת הארגון.

נקודה למבחן

מתקפת SolarWinds היא דוגמה קלאסית ל:

- Attack. Chain Supply
- Movement. Lateral
- (APT). Threat Persistent Advanced
- מתקפה שקשה לזהות באמצעות אנומליות פשוטות.

הלקח המרכזי הוא שלעיתים התוקף המוצלח ביותר אינו זה שמייצר את האנומליה הגדולה ביותר, אלא זה שמצליח להיראות כמו משתמש רגיל.

מדוע מינימום ריבועים מוביל לממוצע ומינימום ערכים מוחלטים מוביל לחציון

שתי תוצאות יסודיות בסטטיסטיקה מסבירות מדוע הממוצע והחציון מופיעים באופן טבעי בבעיות אופטימיזציה.

מינימום ריבועים והממוצע

נתונים:

$$x_1, x_2, \dots, x_n$$

נחפש את הערך a הממזער את סכום ריבועי המרחקים:

$$L(a) = \sum_{i=1}^n (x_i - a)^2$$

נגזור:

$$L'(a) = \sum_{i=1}^n 2(a - x_i) = 2 \left(na - \sum_{i=1}^n x_i \right)$$

נשווה לאפס:

$$L'(a) = 0$$

ולכן:

$$na = \sum_{i=1}^n x_i$$

כלומר:

$$a = \frac{1}{n} \sum_{i=1}^n x_i$$

ולכן:

$$\arg \min_a \sum_{i=1}^n (x_i - a)^2 = \bar{x}$$

כלומר:

הממוצע הוא הנקודה הממזערת את סכום ריבועי המרחקים.

מינימום ערכים מוחלטים והחציון

כעת נבחן:

$$L(a) = \sum_{i=1}^n |x_i - a|$$

פונקציה זו אינה גזירה בכל נקודה, אך ניתן להבין את הפתרון באופן גיאומטרי. אם a נמצא משמאל לחציון, אז רוב הנקודות נמצאות מימין, ולכן הזזה קטנה ימינה מקטינה את הסכום. אם a נמצא מימין לחציון, אז רוב הנקודות נמצאות משמאל, ולכן הזזה שמאלה מקטינה את הסכום.

שיווי המשקל מתקבל בדיוק כאשר מספר הנקודות משמאל ל- a שווה (בקירוב) למספר הנקודות מימינו.
זוהי בדיוק הגדרת החציון.
לכן:

$$\arg \min_a \sum_{i=1}^n |x_i - a| = \text{Median}(X)$$

כלומר:

החציון הוא הנקודה הממזערת את סכום המרחקים המוחלטים.

אינטואיציה

□ ריבוע מעניש מאוד על טעויות גדולות, ולכן חריגים מושכים את הפתרון לכיוונם. כתוצאה מכך מתקבל הממוצע.

□ ערך מוחלט מעניש בצורה ליניארית, ולכן חריגים משפיעים הרבה פחות. כתוצאה מכך מתקבל החציון.

זו הסיבה ש:

□ הממוצע וסטיית התקן רגישים לחריגים.

□ החציון ו-MAD עמידים לחריגים.

נקודה למבחן

$$\arg \min_a \sum (x_i - a)^2 = \text{Mean}$$

$$\arg \min_a \sum |x_i - a| = \text{Median}$$

אלו שתיים מהתוצאות החשובות ביותר המחוברות בין אופטימיזציה, סטטיסטיקה ולמידת מכונה.